

Let `strong_order` Truly Be a Customization Point!

Gašper Ažman

Document #: P0891R0
Date: 2018-02-10
Audience: Library Working Group
Reply-to: Gašper Ažman, gasper.azman@gmail.com

Contents

1	Abstract	1
2	Status of this paper	1
3	Problem Description	2
4	Current Status	2
5	Code Example	2
6	Proposal	4
6.1	Make <code>strong_order</code> An Explicit Customization Point	4
6.2	Remove the <code>iec559</code> Exception (point 1.1)	4
6.3	Fix The Rules for Synthesis of Weaker Algorithms	4
6.4	Designate The Rest of the Algorithms to be Customization Points Too	5
6.5	Alternative	5
7	Exposition: The Natural and Default Orderings	5
7.1	On Compatibility Between the Natural and Default Orderings	5
8	Acknowledgments	6

1 Abstract

This paper is a defect-report to a library extension that has been voted into the working draft as part of P0768R1[1].

In communication with Herb Sutter and Walter Brown, it was made clear that all 5 algorithms mentioned in this paper are meant to be customization points.

The first part of this paper is a defect report. The sencond part of the paper is an elucidation of what the semantics of the `strong_order` customization point should be.

2 Status of this paper

The wording for the entire fix is not provided in this paper, and shall be written if this paper receives support and further guidance on direction of how we want to handle customization points.

It then highlights one problem with the current wording that makes it unsuitable for that purpose, and highlights also a slight inelegance with the current integration of `iec559` types.

3 Problem Description

The current C++ standard does not have an explicitly designated customization point for providing a *default ordering*¹. *Elements of Programming* uses `less<T>::operator()` for this purpose, as does the global order for pointers; but in the wake of `operator <=>`, `less<T>` is missing features, such as computing equality without calling it twice. It has also failed to get adoption for this purpose throughout the years, perhaps exactly due to the missing features.

The wording of point 1.1 of the `strong_order` algorithm suggests that `strong_order` is finally this missing customization point for specifying a default ordering for types whose natural ordering is not strong and total, since it does exactly that for the `iec559` types.

The issue is that the rest of the points make this function rather unsuitable for use as a customization point, since the language explicitly makes it not SFINAE-friendly. In the event that it cannot be synthesized, it is marked as *deleted*, and not as *"shall not participate in overload resolution"*.

4 Current Status

For reference, the current specification for the `strong_order` algorithm looks like this:

```
template<class T>
constexpr strong_ordering strong_order(const T& a, const T& b);
```

1. Effects: Compares two values and produces a result of type `strong_ordering`:
 - 1.1. If `numeric_limits<T>::is_iec559` is true, returns a result of type `strong_ordering` that is consistent with the `totalOrder` operation as specified in ISO/IEC/IEEE 60559.
 - 1.2. Otherwise, returns `a <=> b` if that expression is well-formed and convertible to `strong_ordering`.
 - 1.3. Otherwise, if the expression `a <=> b` is well-formed, then the function shall be defined as deleted.
 - 1.4. Otherwise, if the expressions `a == b` and `a < b` are each well-formed and convertible to `bool`, returns `strong_ordering::equal` when `a == b` is true, otherwise returns `strong_ordering::less` when `a < b` is true, and otherwise returns `strong_ordering::greater`.
 - 1.5. Otherwise, the function shall be defined as deleted.

5 Code Example

Let me illustrate on a trivial example. Say we have a template struct representing the gaussian integers, with a *"natural order"*² defined by the manhattan distance from $0 + 0i$. This struct still defines a `strong_order` to model *Regular*³.

```
1 namespace user {
2     template <typename T>
3     struct gaussian {
```

¹See the Exposition section for the definitions and discussion of orderings

²There is no natural order on gaussian integers, but humor this example, please.

³The *Elements of Programming* concept, not the ISO C++ *Regular*, which is weaker.

```

4     static_assert(std::is_integral_v<T>);
5     T re;
6     T im;
7
8     constexpr std::strong_equality operator==(gaussian const& other) const {
9         return re == other.re && im == other.im;
10    }
11    constexpr std::weak_ordering operator<=>(gaussian const& other) const {
12        return (*this == other) ? std::weak_ordering::equal
13            : (abs(*this) == abs(other)) ? std::weak_ordering::equivalent
14            : abs(*this) <=> abs(other);
15    }
16    friend constexpr T abs(gaussian const&) {
17        using std::abs;
18        return abs(re) + abs(im);
19    }
20
21    friend constexpr std::strong_ordering strong_order(gaussian const& x,
22                                                       gaussian const& y) {
23        // compare lexicographically
24        return std::tie(x.re, x.im) <=> std::tie(y.re, y.im);
25    }
26 };
27 }

```

Consider a transparent ordering operator for map:

```

1     struct strong_less
2     template <typename T, typename U>
3     bool operator()(T const& x, U const& y) {
4         using std::strong_order; // use ADL
5         return strong_order(x, y) < 0;
6     }
7     using is_transparent = std::true_type;
8 };

```

Also say we had a type with an implicit conversion to our gaussian:

```

1     template <typename T>
2     struct lazy {
3         std::function<T()> make;
4         operator T() const { return make(); }
5     };

```

This function now fails to compile, because the chosen strong_order is deleted.

```

1     bool exists(lazy<gaussian<int>> const& x,
2               std::set<gaussian<int>, strong_less> const& in) {
3         /* imagine this being a template in both parameters - it's pretty normal */
4         return in.count(x);
5     }

```

The std-provided strong_order is deleted because it cannot be synthesized from gaussian's operator <=>. The reason it is chosen over the friend function, however, is because the standard template matches better than the friend which would require an implicit conversion.

If the std-provided strong_order did not participate in overload resolution, however, this example would work just fine.

6 Proposal

6.1 Make **strong_order** An Explicit Customization Point

Depending on the final direction of the wording on customization points (either the current one, with an emphasis on *shall not participate in overload resolution*, or the one outlined in P0551R0[2], the wording shall differ here.

I am asking LWG for guidance on this subject.

6.2 Remove the **iec559** Exception (point 1.1)

Since this paper adds explicit support for this customization point, the exception can now be implemented using whichever mechanism for customization points is chosen, and this special case moved to that part of the standard. For instance, a "more specialized" template based on a `requires` clause and the `numeric_limits<T>::is_iec559` trait can be added to namespace `std`.

The minimal fix for the current situation would be:

Change point 1.3 to read: *Otherwise, if the expression $a <=> b$ is well-formed,⁴ the function does not participate in overload resolution.*

After the list, add a Note:

*If operator $<=>$ provides an order weaker than strong, this function allows the provision of a default strong order for a user-defined type. In that case, **strong_order** should define a strict, total ordering.*

6.3 Fix The Rules for Synthesis of Weaker Algorithms

The algorithms section contains a few other algorithms:⁵

- `weak_order(const T& a, const T& b)`
- `partial_order(const T& a, const T& b)`
- `strong_equal(const T& a, const T& b)`
- `weak_equal(const T& a, const T& b)`
- `partial_equal(const T& a, const T& b)`

Intuitively, one would expect that if `strong_order` is available, then so are `strong_equal`, `weak_order` and `partial_order` (with `weak_equal` and `partial_equal` being consequences of those). The current situation seems to provide for that by accident⁶, with no explicit reference to this fact.

However, if `strong_order` is the customization point for a default order that *may* be stronger than the order on operator $<=>$, then the above expectation may no longer hold for such types (you might have `strong_order` but not `weak_order`, for instance).

The fix-up for each of the sections describing the above primitives would be to insert, after point x.1 (which describes the algorithm in terms of $<=>$) the automatic fallback to a call to `strong_order`, if it is resolvable through an unqualified call (thus enabling argument-dependent lookup).

⁴Note: point 1.2 already takes care of the case where $<=>$ provides a strong (and thus valid default) order.

⁵Not to be confused with the types of their results; those end in `-ing`: `strong_ordering`, `weak_ordering` etc.

⁶the rules for those algorithms are identical but for the `iec559` exception in `strong_order`; since floating-point types possess `operator<` and `operator==`, they enable the synthesis of all those algorithms.

6.4 Designate The Rest of the Algorithms to be Customization Points Too

They were intended to be customization points. There should be a *Remark* making that clear in every section. However, for the rest of the algorithms, their specific intended use is not quite as clear as the usecase for `strong_order`.

6.5 Alternative

If the purpose of `strong_order` is not to provide a default ordering for types, the `iec559` exception should be removed from the wording, and a different customization point (perhaps called `total_order`) added for the express purpose of providing an arbitrary total order on the entire domain of a type.

7 Exposition: The Natural and Default Orderings

Obviously, there are many reasons for sorting. However, this paper is chiefly concerned with the division between the *natural ordering* and the *default total ordering* as required for *Regular* types by Stepanov and McJones in their seminal work *Elements of Programming* ([3], page 62, section 4.4).

The **natural ordering** is the ordering that makes semantic sense for a type. This is the ordering that operator `<=>` and its library extensions are tailor-made for: not every type is ordered (or even equality-comparable), and when a type supports an ordering, it might be strong, partial, or weak.

We use these orderings when we need them to make sense - heaps, scheduling tasks by topological sorts, various displays for users, etc. Not all value types have a natural ordering, because not all types are ordered. The gaussian integers are one such type.

The **default ordering**⁷ is the strongest ordering that a type admits. Its equality is defined by value-substitutability, and unequal elements must be ordered; it is always strong and total, and might not make semantic sense.

According to *Elements of Programming*, every *Regular* type should provide a default ordering.

A type with a default ordering is far more useful than one without; ordering enables the use of tree-based containers (i.e. `map`, `set`), and algorithms based on sorted data (`unique`, the various set algorithms, and the various versions of binary search) – and this is just the tip of the iceberg. The only requirement for the above is having a total strong ordering - what the ordering *means* is utterly irrelevant.

The lexicographic ordering of the gaussian integers is a good example of a default ordering.

Another excellent example is `float` – its various NaNs and infinities are not ordered, which is why its natural ordering is not suitable as a default ordering. However, `iec559` defines a total strong ordering for those values, thus enabling the uses outlined above.

7.1 On Compatibility Between the Natural and Default Orderings

Elements of Programming specifies that for types where the natural and default orderings differ, the default ordering should be compatible with the natural one: that is, if `a` and `b` are comparable and compare unequal under `<=>`, the default order produces the same result (less or greater).

However, requiring this in the language of the standard library as a mandatory semantic constraint seems like a bad idea.

For instance, if one takes the gaussian integers ordered by the manhattan-distance to zero (sum of absolute values of the two components), the compatible total order (a lexicographic ordering of every equivalence class) is far slower to compute than the simple lexicographic one.

⁷The name comes from *Elements of Programming*

Furthermore, if needed, a compatible total order can always be achieved on the fly by comparing with the natural order first - if the result is less or greater, keep the result - otherwise, fall back on the default ordering.

8 Acknowledgments

I would like to thank

Roger Orr for bringing this to my attention;

Thomas Köppe for his valuable comments, review, and most of all some extremely clear and laconic wording;

Sam Finch for *thoroughly* breaking my examples, some example code, great substantive comments, and pointing out that the current definition actually breaks types that define a partially-ordered set of comparison operators;

Richard Smith for further fixing my example in light of Concepts, and example code.

Herb Sutter and Walter Brown for providing guidance on customization points.

Louis Dionne for great comments on the structure of the paper and how to bring the focus where it needs to be.

Thanks!

References

- [1] Walter E. Brown. “Library Support for the Spaceship (Comparison) Operator”. In: *Post-Albuquerque Mailing* (2017). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0768r1.pdf>.
- [2] Walter E. Brown. “Thou Shalt Not Specialize std Function Templates”. In: (2017). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0551r0.pdf>.
- [3] Alexander Stepanov and Paul McJones. *Elements of Programming*. 1st. Addison-Wesley Professional, 2009. ISBN: 032163537X, 9780321635372.